

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)

Факультет «Информатика и вычислительная техника»
Кафедра «Кибербезопасность информационных систем»

Практическая работа №2

по дисциплине: «Программирование алгоритмов защиты информации»
На тему «Калькулятор на эллиптических кривых»

Выполнил обучающийся гр. ВКБ43
Ковалев Данил Петрович

Проверил:
Савельев В.А.

Ростов-на-Дону
2026

Тема и цель работы

Тема: вычисления в группе точек эллиптической кривой над простым полем. Цель — освоить модульную арифметику и операции с точками, исследовать группу и продемонстрировать обмен ключами.

Постановка задачи

Преподавателем задано название без дополнительного условия и варианта. Принята постановка: создать консольный калькулятор для невырожденных кривых Вейерштрасса над Fp, p > 3.

- Проверять параметры кривой и принадлежность точки.
- Вычислять −P, P+Q, P−Q, 2P, kP и kP+mQ.
- Исследовать точки, порядки, подгруппы и таблицу сложения.
- Показывать ECDH и перебор дискретного логарифма.
- Проверить особые случаи и оформить результаты.

Используемые средства

Средство	Назначение
rustc 1.96.0 (ac68faa20 2026-05-25)	Rust edition 2024; минимальная заявленная версия 1.85
clap 4; tracing; color-eyre; thiserror	CLI, журнал операций, диагностические ошибки
cargo test; proptest; Clippy; rustfmt	Автоматизированная проверка и контроль качества
Python; python-docx; freeze	Независимая сверка и генерация отчёта

Таблица 1 — программные средства

Теоретические сведения

Поле и эллиптическая кривая

Поле Fp состоит из классов вычетов 0, …, p−1 по простому модулю p. Все действия с координатами выполняются по модулю p. Деление на ненулевое d означает умножение на обратный элемент d^{-1}. В данной реализации он вычисляется по малой теореме Ферма: $d^{-1} = d\text{^}(p-2) \bmod p$.

$$E(Fp): y^2 \equiv x^3 + ax + b\ (mod\ p), p > 3$$

$$\Delta = -16(4a^3 + 27b^2) \not\equiv 0\ (mod\ p)$$

Условие $\Delta \neq 0$ исключает особые точки кривой. К множеству конечных решений (x,y) добавляется точка на бесконечности O. Именно это множество с законом сложения образует абелеву группу [1, § 2.2.1; 2, § 3].

Сложение и удвоение точек

Для $P=(x_1,y_1)$, $Q=(x_2,y_2)$, $P \neq Q$ и $x_1 \neq x_2$ применяется секущая. При P=Q и $y_1 \neq 0$ применяется касательная. Коэффициент λ вычисляется в поле:

$$\lambda = (y_2 - y_1)(x_2 - x_1)^{-1}\ mod\ p\ \ (P \neq Q)$$

$$\lambda = (3{x_1}^2 + a)(2y_1)^{-1}\ mod\ p\ \ (P = Q)$$

$$x_3 = \lambda^2 - x_1 - x_2\ mod\ p$$

$$y_3 = \lambda(x_1 - x_3) - y_1\ mod\ p$$

Особые случаи рассматриваются до деления: P+O=P; O+O=O; $-P=(x_1,-y_1 \bmod p)$; P+(−P)=O. Если $y_1=0$, то 2P=O. Вычитание определяется как P−Q=P+(−Q) [1, § 2.2.1].

Скалярное умножение и структура группы

Скалярное умножение kP означает повторное сложение точки. Для k=0 результат равен O, для отрицательного k используется противоположная точка. Двоичный алгоритм double-and-add последовательно обрабатывает биты |k|: для установленного бита прибавляет текущую точку к накопителю и удваивает текущую точку перед переходом к следующему разряду [1, § 2.2.1].

$$k = \Sigma\, b_i 2^i;\ \ kP = \Sigma\, b_i (2^i P)$$

Число групповых операций составляет O(log |k|). В аффинных координатах каждая обычная операция сложения содержит модульное обращение; двоичное возведение в степень для обращения требует O(log p) модульных умножений. Поэтому число модульных умножений при скалярном умножении имеет порядок O(log |k| · log p).

Число точек #E включает O. Порядок точки ord(P) — минимальное положительное n, при котором nP=O. Для O он равен 1. Подгруппа ⟨P⟩ содержит O,P,…,(n−1)P; по теореме Лагранжа ord(P) делит #E. Порядок точки и число точек всей кривой в общем случае различаются.

Для перечисления точек сначала строится таблица квадратов $y^2 \bmod p$, затем для каждого x вычисляется x^3+ax+b и извлекаются соответствующие y. Использование BTreeMap даёт O(p log p) операций работы с таблицей и O(p) памяти; полный двойной перебор x,y потребовал бы $O(p^2)$ проверок.

Поиск порядка выполняется последовательным сложением до первого O. Дискретный логарифм Q=kP ищется начиная с k=0. Если подгруппа замкнулась, а Q не встретилась, решения нет; если исчерпан лимит, выводится только сообщение об исчерпании лимита. Строится полная таблица сложения для малой группы.

Программная реализация

Отдельный crate practice_02 включён в Cargo workspace. Слой domain содержит чистую математику; application организует вычисления и журналирование; presentation разбирает CLI. Внешних портов и инфраструктурных адаптеров у вычислений нет.

```
crates/practice_02/
  src/domain/field.rs  — поле и проверка простоты
  src/domain/curve.rs  — точки и групповые операции
  src/application/     — сценарии калькулятора
  src/presentation/    — clap CLI
  tests/domain.rs      — математические проверки
  tests/cli.rs         — сквозные проверки команд
```

Листинг 1 — структура программы

Точка представлена перечислением Infinity либо Affine{x,y}. Проверка параметров выполняется при создании Curve, проверка принадлежности — перед каждой публичной операцией над точкой. Числа со знаком на входе нормализуются в 0,…,p−1. Скаляр i128::MIN обрабатывается через unsigned_abs без переполнения знакового типа.

Модуль хранится в u64. Произведение двух u64 помещается в u128: $(2^{64}-1)^2 < 2^{128}$. Поэтому сложение и умножение перед редукцией выполняются в u128, а вычитание — через i128. Готовые криптографические библиотеки и библиотеки длинной арифметики не используются.

```rust
    pub fn add(self, a: u64, b: u64) -> u64 {
        ((u128::from(a) + u128::from(b)) % u128::from(self.modulus)) as u64
    }

    pub fn sub(self, a: u64, b: u64) -> u64 {
        self.normalize(i128::from(a) - i128::from(b))
    }

    pub fn mul(self, a: u64, b: u64) -> u64 {
        mul_mod(a, b, self.modulus)
    }

    pub fn pow(self, base: u64, exponent: u64) -> u64 {
        pow_mod(base, exponent, self.modulus)
    }
```

Листинг 2 — безопасная модульная арифметика

Алгоритм сложения точек

Внутренний метод add_valid вызывается только после проверки точек или для результатов уже корректных операций. Сначала обрабатываются O и противоположные точки, затем выбирается формула λ. Обратный элемент нуля не вычисляется.

```
    fn add_valid(self, left: Point, right: Point) → Result<Point, Error> {
        let (Point::Affine { x: x1, y: y1 }, Point::Affine { x: x2, y: y2 }) = (left, right) else {
            return Ok(if left == Point::Infinity { right } else { left });
        };
        let f = self.field;
        if x1 == x2 && f.add(y1, y2) == 0 {
            return Ok(Point::Infinity);
        }
        let (numerator, denominator) = if left == right {
            (f.add(f.mul(3, f.mul(x1, x1)), self.a), f.mul(2, y1))
        } else {
            (f.sub(y2, y1), f.sub(x2, x1))
        };
        let slope = f.mul(numerator, f.inverse(denominator)?);
        let x3 = f.sub(f.sub(f.mul(slope, slope), x1), x2);
        let y3 = f.sub(f.mul(slope, f.sub(x1, x3)), y1);
        Ok(Point::Affine { x: x3, y: y3 })
    }
```

Листинг 3 — реализация закона сложения

Алгоритм скалярного умножения

```rust
    pub fn multiply(self, point: Point, scalar: i128) -> Result<Point, Error> {
        self.validate(point)?;
        let mut base = if scalar < 0 {
            self.negate_valid(point)
        } else {
            point
        };
        let mut bits = scalar.unsigned_abs();
        let mut result = Point::Infinity;
        while bits != 0 {
            if bits & 1 != 0 {
                result = self.add_valid(result, base)?;
            }
            bits >>= 1;
            if bits != 0 {
                base = self.add_valid(base, base)?;
            }
        }
        Ok(result)
    }
```

Листинг 4 — двоичный double-and-add

Проверка простоты p использует детерминированный Miller–Rabin с основаниями 2, 325, 9375, 28178, 450775, 9780504, 1795265022. Этот набор покрывает диапазон $n < 2^{64}$ [3]. Он позволяет проверять большие учебные модули без пробного деления до $\sqrt{p}$.

Ручная проверка контрольного примера

Выберем E: $y^2=x^3+2x+2 \bmod 17$ и P=(5,1). Проверка принадлежности: $1^2=1$, а $5^3+2\cdot5+2=137\equiv1 \bmod 17$. Условие невырожденности: $4\cdot2^3+27\cdot2^2=140\equiv4$, $\Delta\equiv-16\cdot4\equiv4\neq0$.

Для удвоения: числитель $3\cdot5^2+2=77\equiv9$; знаменатель $2\cdot1=2$, обратный элемент $2^{-1}\equiv9$. Тогда $\lambda\equiv9\cdot9\equiv13$; $x_3\equiv13^2-5-5\equiv6$; $y_3\equiv13(5-6)-1\equiv3$. Следовательно, 2P=(6,3).

Для P+(6,3): $\lambda\equiv(3-1)/(6-5)\equiv2$, $x_3\equiv4-5-6\equiv10$, $y_3\equiv2(5-10)-1\equiv6$. Поэтому 3P=(10,6). Обратная точка −P=(5,16), а P+(−P)=O.

Вычисление	Ожидаемый результат
2P; 3P; 7P	(6,3); (10,6); (0,6)
−3P; 0P; 19P	(10,11); O; O
2P+3Q, Q=2P	8P=(13,7)
#E; ord(P)	19; 19
ECDH: a=5, b=7	A=(9,16); B=(0,6); S=35P=16P=(10,11)

Таблица 2 — контрольные значения

Сборка и команды запуска

```
cargo build -p practice_02
just practice-02 --help
just practice-02 demo
```

```
just practice-02 info
just practice-02 check 5,1
just practice-02 neg 5,1
just practice-02 add 5,1 6,3
just practice-02 sub 5,1 6,3
just practice-02 double 5,1
just practice-02 mul 7 5,1
just practice-02 mul -- -3 5,1
just practice-02 linear 2 5,1 3 6,3
just practice-02 inverse 2
```

Листинг 5 — арифметические операции

```
just practice-02 points
just practice-02 count
just practice-02 order 5,1
just practice-02 subgroup 5,1
just practice-02 dlog 5,1 0,6
just practice-02 ecdh 5,1 5 7
just practice-02 --p 5 --a 1 --b 0 table
```

Листинг 6 — исследование группы и ECDH

Для произвольной кривой задаются --p, --a, --b. Без этих опций используются p=17,a=2,b=2. Команда demo намеренно показывает фиксированный набор примеров на этой кривой. Координаты записываются x,y, нейтральный элемент — O. RUST_LOG=off отключает журнал, RUST_LOG=debug включает диагностику. Вычисленный результат печатается в stdout, ошибки — в stderr.

Результаты арифметических операций

Полученные суммы, разности и кратные совпали с ручными контрольными значениями. Включены отрицательный и нулевой скаляры, а также сумма противоположных точек.

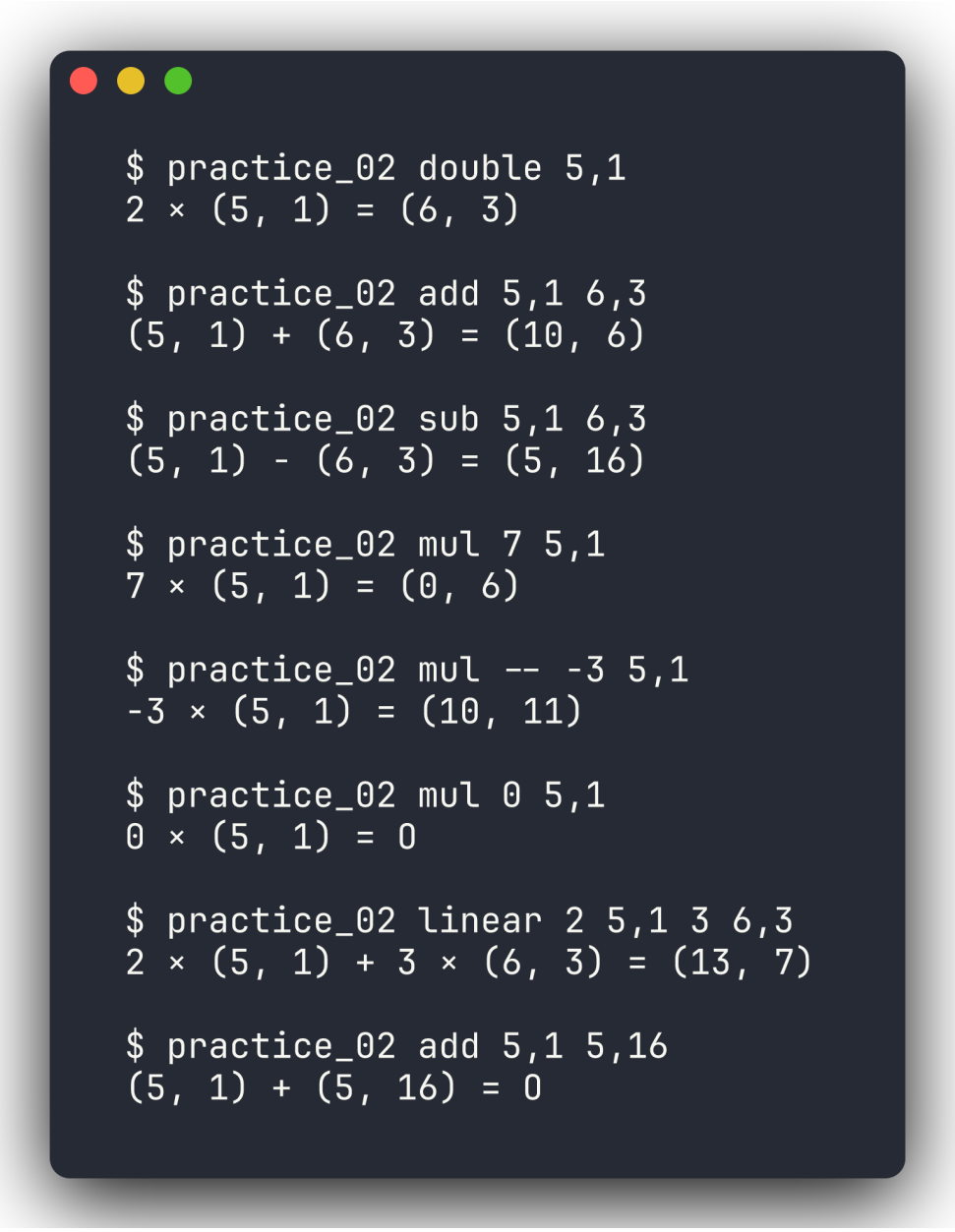


Рисунок 1 — реальные запуски основных операций

Точки демонстрационной кривой

В списке 19 элементов, включая O. На этой кривой точка (5,1) имеет порядок 19 и порождает всю группу.

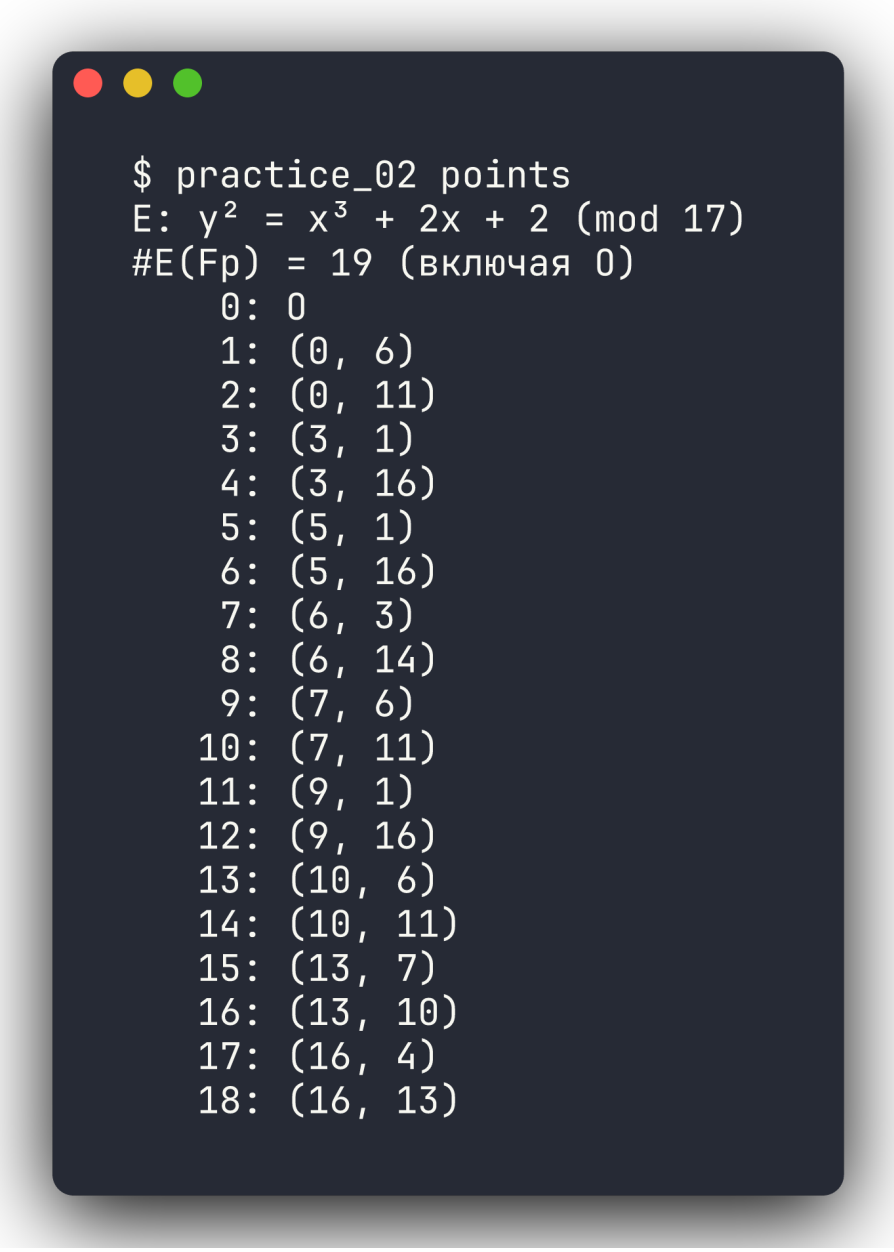


Рисунок 2 — все точки E17(2,2)

Порядок и циклическая подгруппа

Каждая из 19 точек встречается в списке кратных ровно один раз, начиная с 0P=O; следующий элемент 19P снова равен O.

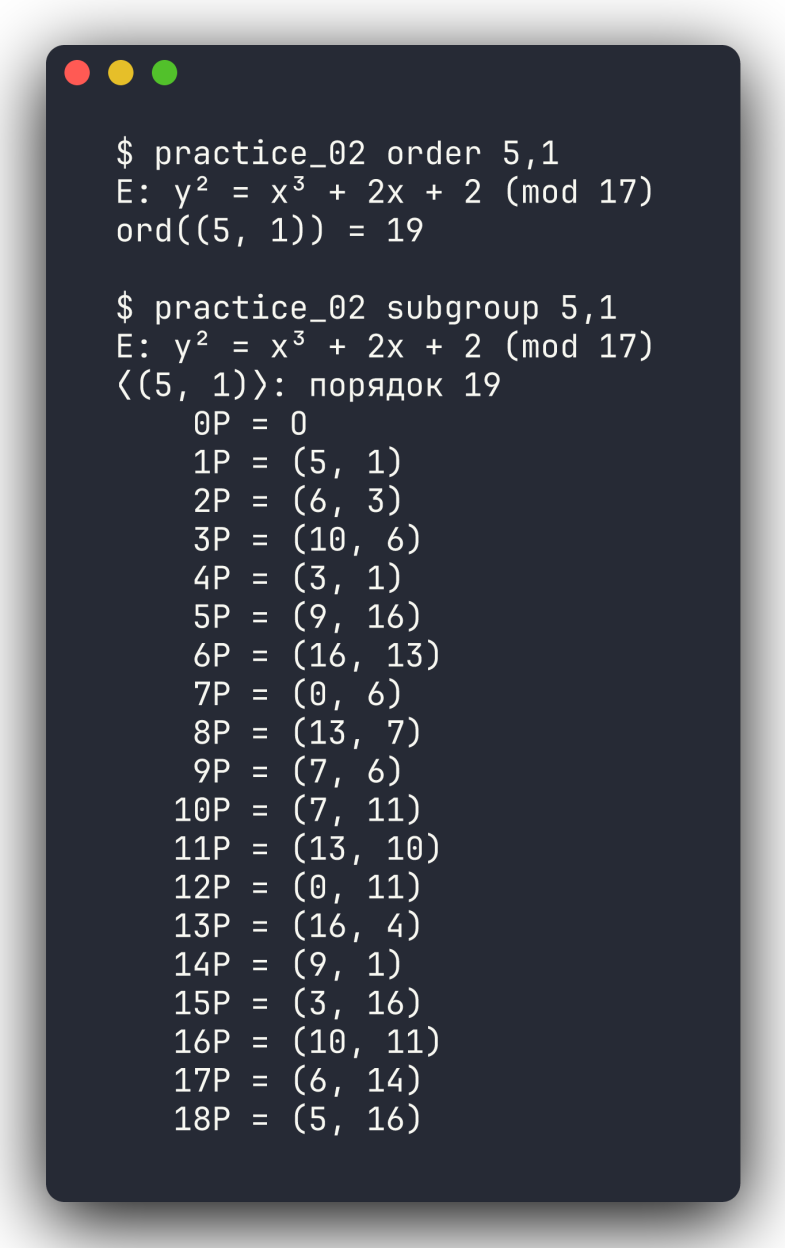
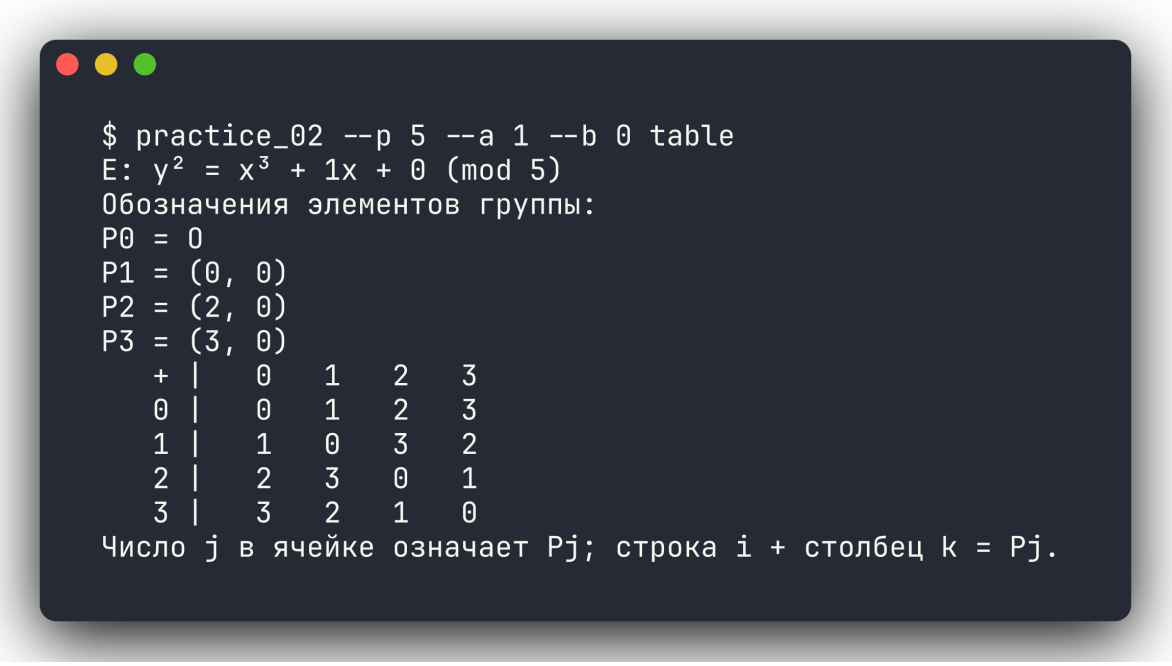


Рисунок 3 — порядок точки и её кратные

Таблица сложения малой группы

Группа состоит из O,(0,0),(2,0),(3,0). Каждая ненулевая точка имеет порядок 2, а число точек всей кривой равно 4. Этот пример показывает различие двух понятий порядка.



```
$ practice_02 --p 5 --a 1 --b 0 table
E: y² = x³ + 1x + 0 (mod 5)
Обозначения элементов группы:
P0 = O
P1 = (0, 0)
P2 = (2, 0)
P3 = (3, 0)
   + |   0   1   2   3
   0 |   0   1   2   3
   1 |   1   0   3   2
   2 |   2   3   0   1
   3 |   3   2   1   0
Число j в ячейке означает Pj; строка i + столбец k = Pj.
```

Рисунок 4 — таблица E5(1,0)

Дискретный логарифм и обмен ECDH

$$A = aG;\ B = bG;\ S_{\text{а}} = aB = abG = bA = S_{\text{в}}$$

Участники используют одну точку G и заданные скаляры a,b. Совпадение общих точек следует из свойств скалярного умножения. В программе выводятся публичные и общие точки; случайная генерация секретов и получение сеансового ключа через KDF не выполняются [2, § 4].

Для Q=(0,6) получено k=7. При a=5,b=7 обе стороны обмена вычислили одну общую точку (10,11). Перебор дискретного логарифма здесь возможен благодаря малому размеру группы.

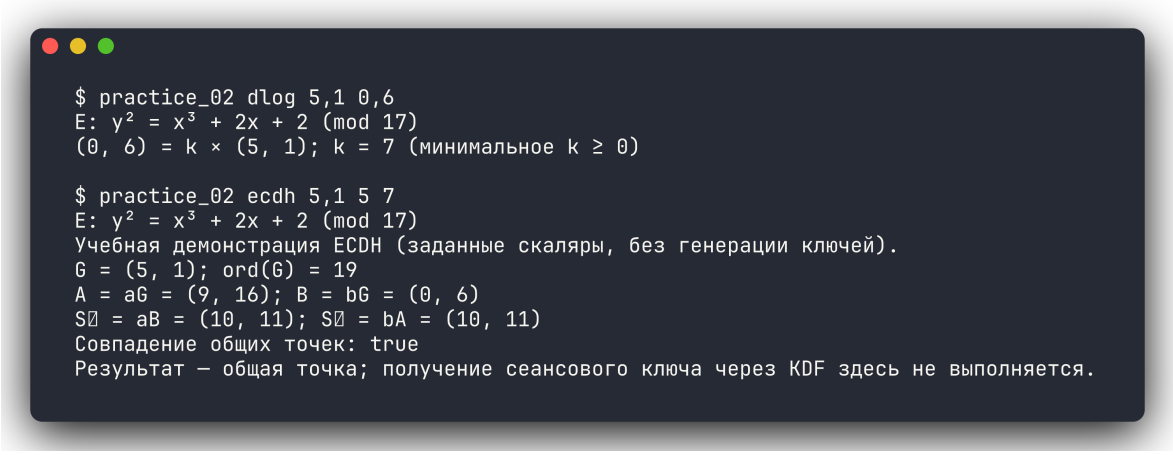


Рисунок 5 — поиск k и совпадение общих точек

Проверка ошибочных входных данных

Составной модуль, вырожденная кривая и арифметика с посторонней точкой завершаются ошибкой. Команда check для посторонней точки корректно возвращает false. Превышение лимита полного перебора обозначается явно. На рисунке показаны сообщения и коды завершения; полная диагностика сохранена в журнале 06_errors.log.

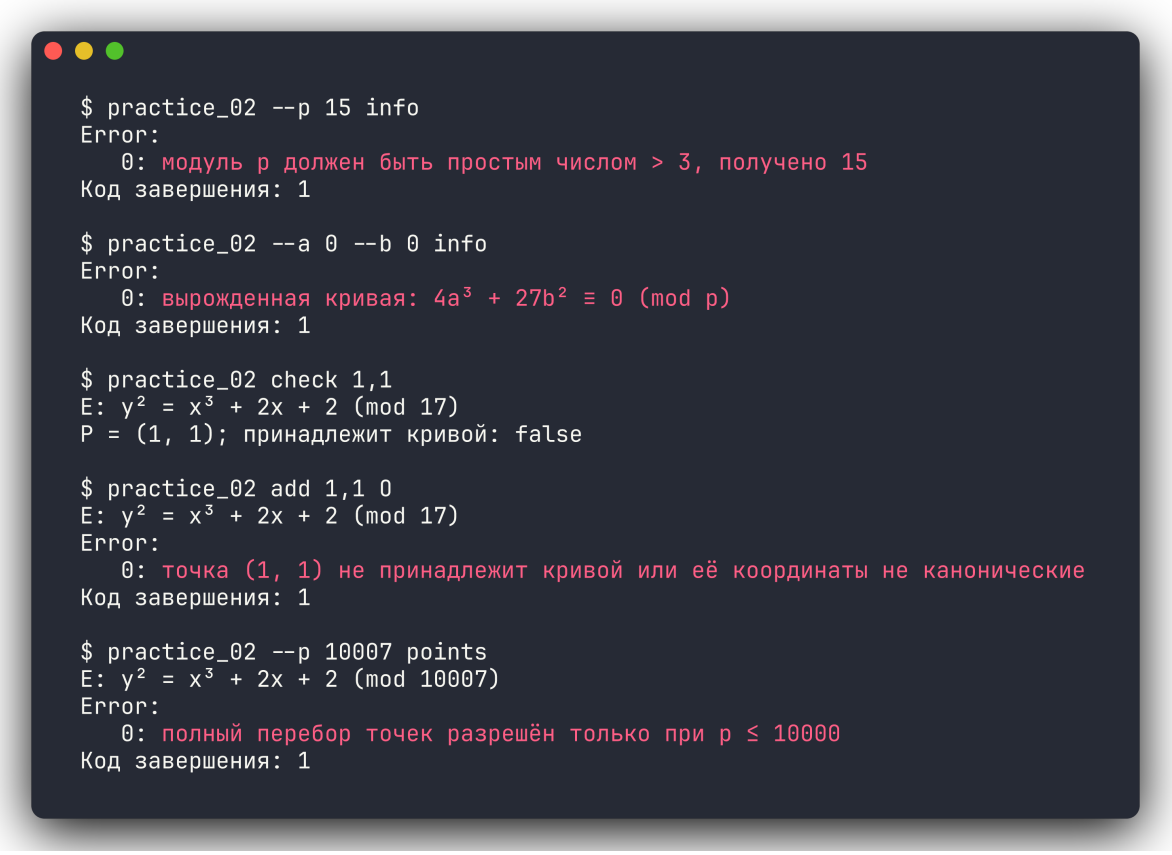


Рисунок 6 — фрагменты фактических сообщений об ошибках

Проверка большого модуля

Обычные операции с точками поддерживают весь диапазон u64 для p. Ограничение p≤10000 относится к полному перечислению, а не к арифметике.

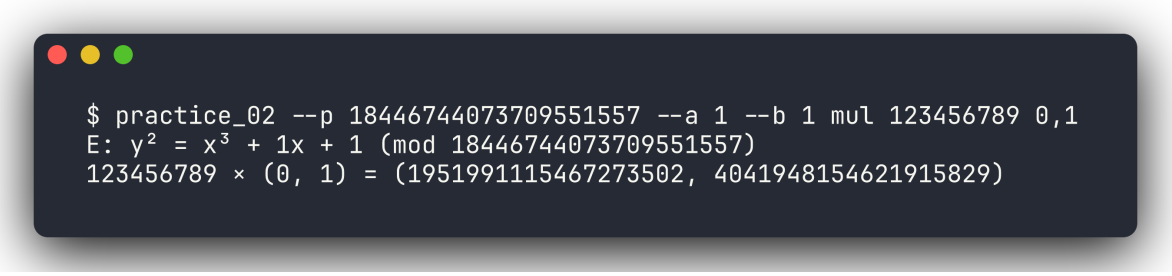


Рисунок 7 — арифметика у верхней границы u64

Контроль качества и тестирование

Проверено 2026-10-31T12:43:29+03:00. just ci: код 0; форматирование, Clippy и тесты workspace прошли успешно. Практика 2: 23 теста, 0 ошибок.

Группа проверок	Содержание
Ручные векторы	18 ненулевых кратных G и замыкание 19G=O; пример ECDH
Групповые свойства	Все 19^3=6859 троек для ассоциативности; proptest: коммутативность, обратимость, распределительность и композиция скаляров
Независимый перебор	Прямая проверка уравнения двойным циклом на нескольких кривых; граница Хассе и делимость порядков
Граничные значения	u64-модуль около 2^{64}; i128::MIN/MAX; ноль, O, y=0; псевдопростые числа
CLI	Все подкоманды; своя кривая; неверные аргументы и ненулевые коды ошибок
Независимая сверка	33 вычисления на трёх кривых с Python big integers и другим способом модульного обращения

Таблица 3 — выполненные проверки

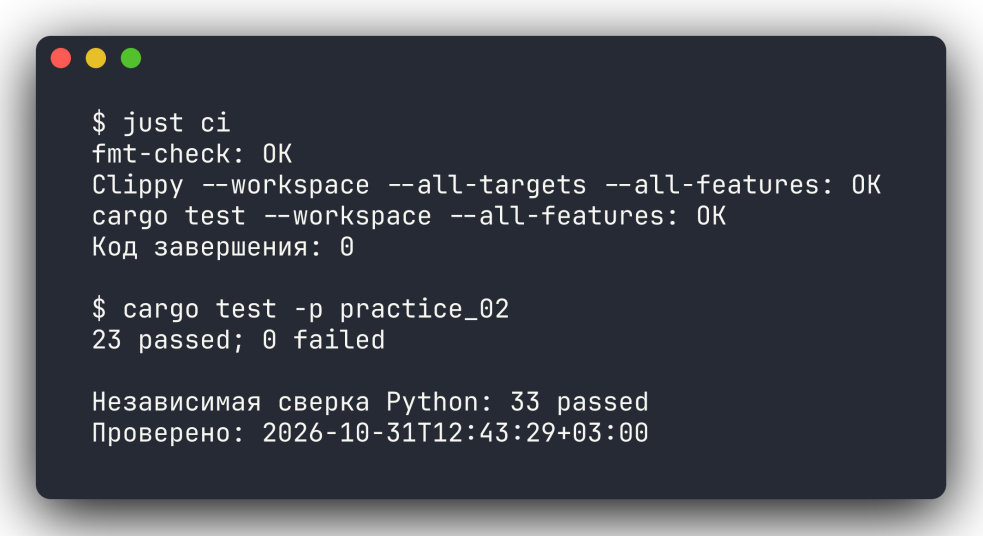


Рисунок 8 — итог обязательных проверок

Полные журналы и независимые сверки: docs/reports/practice_02/evidence. Все итоги получены из фактических запусков.

Ограничения и обсуждение решения

- Поддерживается короткая форма Вейерштрасса над простым полем характеристики >3; расширения F_{2^m} и стандартные 256-битные кривые не входят в реализацию.
- Полное перечисление точек разрешено при $p \leq 10000$, таблица — до 32 точек, поиск порядка/подгруппы/логарифма — до 100000 сложений. Ограничения предотвращают длительные переборы и чрезмерный вывод.
- Скалярное умножение двоичное, координаты аффинные. Проективные координаты, алгоритм Шуфа и быстрые методы дискретного логарифма не реализованы.
- ECDH — учебное вычисление общих точек с заданными скалярами. Постоянное время, случайная генерация ключей, KDF и прикладное шифрование отсутствуют. Код не предназначен для защиты реальных данных.

Умножение двух точек и обычное деление точки на точку не являются стандартными групповыми операциями. Выражение kP означает умножение на целое, а задача найти k из Q=kP — дискретный логарифм. В расширенном калькуляторе эти понятия представлены отдельными командами.

Контрольные вопросы для самопроверки

Вопросы сформулированы для объяснения выполненной реализации; отдельный список вопросов преподавателем не предоставлен.

Вопрос 1. Почему модуль должен быть простым?

Ответ. В Fp каждый ненулевой элемент имеет обратный. При составном модуле кольцо вычетов может содержать делители нуля и обычные формулы сложения не обеспечивают заявленную группу.

Вопрос 2. Для чего нужна точка O?

Ответ. Она является нейтральным элементом, результатом суммы противоположных точек и результатом нулевого скалярного умножения.

Вопрос 3. Чем отличается порядок точки от числа точек?

Ответ. Порядок точки — размер порождённой ею циклической подгруппы. Число точек — размер всей группы. Первый делит второй, но может быть меньше.

Вопрос 4. Почему нельзя делить координаты обычным целочисленным делением?

Ответ. Деление в поле — умножение на модульный обратный элемент. Например, 1/2 mod 17 равно 9, потому что $2 \cdot 9 \equiv 1$.

Вопрос 5. Почему double-and-add эффективнее последовательного сложения?

Ответ. Он обрабатывает двоичные разряды скаляра и требует порядка $\log|k|$ групповых операций вместо порядка $|k|$.

Вопрос 6. Почему в ECDH совпадают результаты участников?

Ответ. $A=aG$ и $B=bG$, следовательно $aB=(ab)G=(ba)G=bA$. Общая точка затем требует отдельного преобразования в сеансовый ключ в полноценном протоколе.

Выводы

Выполнена практическая работа по построению консольного калькулятора на эллиптических кривых. Реализованы модульная арифметика, проверка параметров, закон сложения и двоичное скалярное умножение, перечисление точек и исследование группы. На конкретных примерах продемонстрированы дискретный логарифм и совпадение общих точек ECDH.

Результаты вычислений совпали с ручными примерами и независимым эталоном на Python. Автоматические проверки подтвердили корректность особых случаев, свойств группы и арифметики у границы u64. Подготовлены инструкции запуска, воспроизводимые журналы и отчёт с фрагментами фактических исходников.

Список использованных источников

1. Certicom Research. Standards for Efficient Cryptography 1 (SEC 1): Elliptic Curve Cryptography. Version 2.0, 2009. § 2.2.1. URL: https://www.secg.org/sec1-v2.pdf.

2. McGrew D., Igoe K., Salter M. Fundamental Elliptic Curve Cryptography Algorithms. RFC 6090, 2011. §§ 2–4. URL: https://www.rfc-editor.org/rfc/rfc6090.

3. Izykowski W., Worley S. Deterministic variants of the Miller–Rabin primality test. Набор семи оснований Jim Sinclair для $n<2^{64}$. URL: https://miller-rabin.appspot.com/.